

integration_sec07

7. Phase 6: Enterprise UX and User-Facing Features

Phase 6 represents the culmination of the LLMsVerifier integration effort—the layer where the underlying scoring, routing, and verification infrastructure becomes tangible and accessible to end users. Where Phases 1 through 5 established the foundational scoring engine, provider management, routing intelligence, quality assurance mechanisms, and CI/CD integration, Phase 6 channels all of that computational work into polished, enterprise-grade user experiences. The guiding principle of this phase is **transparency through instrumentation**: every model score, routing decision, quality metric, and fallback event is surfaced to the user in a way that builds trust and enables informed decision-making. This chapter covers four interconnected workstreams: the model selection user experience with rich score visualization, a real-time model status dashboard powered by WebSocket streaming, a batch translation system with multi-model orchestration, and a closed-loop translation quality feedback mechanism that continuously refines model capability scores based on real user interactions.

The architecture of Phase 6 is designed to serve two distinct personas simultaneously. **End users** (translators, localization engineers, and enterprise customers) need intuitive interfaces that communicate which model is being used, why it was selected, and how well it performed. **Platform operators** (SRE teams, model governance officers, and product managers) need dashboards that expose system health, model degradation patterns, routing efficiency metrics, and quality trend analysis. The components described in this chapter satisfy both audiences without compromising the clarity needed by either. All UI components are built in TypeScript/React for the web interface and Go for backend orchestration, maintaining consistency with the HelixTranslate technology stack established in prior phases.

7.1 Model Selection User Experience

The model selection interface is the primary touchpoint where users interact with the LLMsVerifier scoring system. It transforms abstract numerical scores into actionable visual information, enabling users to choose translation models with confidence. The design follows a card-based metaphor where each available model is represented as a self-contained card containing all information necessary for an informed selection decision. The card layout is hierarchical: provider identity and model name occupy

the header row, the overall verification score is rendered as a prominent color-coded badge, capability tags communicate domain expertise, and bottom-row metrics convey operational characteristics such as cost tier and speed classification.

7.1.1 Score Badge Visualization

The score badge is the most visually dominant element of the model card because it communicates the single most important piece of information: the model's verified quality score on a 0–10 scale. The color coding scheme follows a traffic-light metaphor with five distinct tiers. Scores of 9.0 and above render in green (#22C55E), indicating exceptional quality suitable for the most demanding translation tasks such as legal contract localization and medical documentation. Scores between 7.5 and 8.9 render in blue (#3B82F6), denoting strong performance appropriate for professional translation workflows. The amber tier (#EAB308, scores 6.0–7.4) signals acceptable quality for general-purpose content where cost efficiency may outweigh absolute accuracy. Scores between 4.0 and 5.9 render in orange (#F97316), flagging models that should be used cautiously and only for low-stakes content or as fallback options. Any model scoring below 4.0 displays in red (#EF4444), serving as a visual warning that the model has failed verification criteria and should not be used for production translation work. The numeric value is always displayed with one decimal place of precision (e.g., "8.3" rather than "8"), reflecting the granularity of the underlying scoring engine.

7.1.2 Capability Tags and Metadata

Beneath the score badge, each model card renders a horizontal row of capability chips that communicate the model's verified domain competencies. These tags are derived directly from the `capabilities` field of the `VerifiedModel` record populated by the Phase 2 scoring engine. Common tags include `CapDomainLegal`, `CapDomainMedical`, `CapDomainTechnical`, `CapDomainCreative`, and `CapGeneral`. Each tag is rendered as a small, rounded pill-shaped element with a consistent color scheme that corresponds to the domain family. The provider icon (e.g., OpenAI's spiral, Anthropic's constellation, or Google's "G" mark) appears in the header row alongside the model name, providing immediate brand recognition and reinforcing the trust relationship between the user and the model vendor.

7.1.3 Filtering and Sorting Interface

The model selection panel includes a comprehensive filtering system that allows users to narrow the model list based on operational requirements. The provider filter supports multi-select, enabling users to include or exclude specific vendors (for example, excluding all models from a provider undergoing a known outage). The score range slider allows users to set minimum and maximum score thresholds, which is particularly useful when searching for models within a specific quality band. Capability toggles function as a set of checkboxes where users can require that displayed models possess specific domain competencies. The cost tier selector offers four tiers—Free, Standard, Premium, and Enterprise—allowing budget-conscious users to filter by economic constraints.

Sorting options include four dimensions, each supporting ascending and descending order. **Score** (descending) is the default sort, presenting the highest-quality models first. **Speed** sorting orders models by their latency score component, prioritizing

low-latency options for time-sensitive workflows. **Cost** sorting arranges models from least to most expensive, aiding budget optimization. **Recency** sorting surfaces newly verified models, giving users early access to the latest options. Each sort dimension updates the card grid in real time without requiring a page refresh.

The ModelCard component implementation encapsulates all of these concerns into a single reusable React component, as shown in Code Block 1.

Code Block 1: ModelCard component (TypeScript/React)

```
interface ModelCardProps {
  model: VerifiedModel;
  onSelect: (modelId: string) => void;
  isSelected: boolean;
}

const ModelCard: React.FC<ModelCardProps> = ({ model, onSelect,
isSelected }) => {
  const scoreColor = model.score.overall >= 9 ? '#22C55E' :
    model.score.overall >= 7.5 ? '#3B82F6' :
    model.score.overall >= 6 ? '#EAB308' :
    model.score.overall >= 4 ? '#F97316' :
    '#EF4444';

  return (
    <div className={`model-card ${isSelected ? 'selected' : ''}`}
      onClick={() => onSelect(model.modelId)}>
      <div className="model-header">
        <ProviderIcon provider={model.providerId} />
        <span className="model-name">{model.name}</span>
        <span className="score-badge" style={{ backgroundColor:
scoreColor }}>
          {model.score.overall.toFixed(1)}
        </span>
      </div>
      <div className="model-capabilities">
        {model.capabilities.map(cap => (
          <CapabilityTag key={cap} name={cap} />
        ))}
      </div>
      <div className="model-metrics">
        <CostIndicator tier={model.tier} />
        <SpeedIndicator latency={model.score.components.speed} />
        <AvailabilityStatus status={model.status} />
      </div>
    </div>
  );
};
```

The component receives a `VerifiedModel` object (populated from the Phase 2 registry), a selection callback, and a boolean flag indicating selection state. The `scoreColor` computation applies the five-tier color mapping directly, ensuring visual consistency across the entire interface. The `CapabilityTag` sub-component renders each capability as a styled chip, while `CostIndicator`, `SpeedIndicator`,

and `AvailabilityStatus` provide the bottom-row metrics. CSS classes `model-card` and `selected` support hover states, focus rings, and visual distinction for the currently selected model. The click handler propagates the `modelId` upward, triggering the routing logic described in Phase 3.

7.2 Real-Time Model Status Dashboard

Enterprise translation workflows demand real-time visibility into model health. A model that scored 8.5 during the morning verification cycle may degrade to 6.2 by afternoon due to provider-side issues, prompt template drift, or rate-limiting behavior changes. The real-time model status dashboard addresses this operational need by establishing a persistent WebSocket connection between the client browser and the HelixTranslate backend, streaming model status events as they occur. This architecture eliminates the need for periodic polling, reduces server load, and ensures that users see status changes within milliseconds of detection.

7.2.1 WebSocket Event Architecture

The WebSocket endpoint `wss://api.helixtranslate.com/v1/models/stream` emits typed events that conform to the `ModelStatusEvent` interface. Four event types cover the full lifecycle of model state changes. The `score_changed` event fires whenever a model's composite score shifts by more than 0.2 points, carrying both the previous and current scores so that the UI can animate transitions and flag significant changes. The `verification_failed` event indicates that a model has failed a scheduled verification challenge, including a human-readable reason field (e.g., "hallucination in legal terminology test" or "excessive latency on 4K token requests"). The `model_discovered` event announces the registration of a newly verified model, prompting the dashboard to add a new card with a "New" badge. The `model_retired` event signals that a model has been removed from the active pool, typically because its provider has deprecated the endpoint or the model has failed three consecutive verification cycles.

7.2.2 Status Indicator Semantics

Each model card on the dashboard displays a status indicator that aggregates the current score into a high-level health classification. A **green** indicator means the model's score is at or above 7.0, representing healthy operation suitable for all supported translation domains. A **yellow** indicator corresponds to scores between 5.5 and 6.9, signaling degraded performance—usable for general content but requiring monitoring and potentially triggering automatic fallback for high-stakes domains. A **red** indicator indicates a score below 5.5, meaning the model has entered a failing state and should not be used for any production translation until the underlying issue is resolved. A **gray** indicator means the model is currently unavailable (provider downtime, rate limit exhaustion, or maintenance mode), rendering it non-selectable but still visible for transparency.

7.2.3 Degraded Mode Warnings

When a model transitions into yellow or red status, the dashboard emits a non-blocking toast notification to alert the user. If the user has an active translation session using a model that subsequently degrades, the system displays a prominent banner offering one-click migration to the next-best available model. This automatic fallback notification includes the name of the recommended replacement model, its current score, and the estimated latency delta compared to the original selection. The notification respects user preferences: users can opt to remain on the degraded model, auto-switch to the recommended alternative, or configure automatic migration based on score thresholds (e.g., "always switch when score drops below 6.5").

The WebSocket client hook shown in Code Block 2 encapsulates the connection lifecycle and event dispatch logic.

Code Block 2: WebSocket events

```
interface ModelStatusEvent {
  type: 'score_changed' | 'verification_failed' |
        'model_discovered' | 'model_retired';
  modelId: string;
  payload: {
    previousScore?: number;
    currentScore?: number;
    reason?: string;
  };
  timestamp: Date;
}

const useModelStatus = () => {
  const [models, setModels] = useState<VerifiedModel[]>([]);

  useEffect(() => {
    const ws = new WebSocket('wss://api.helixtranslate.com/v1/
                              models/stream');
    ws.onmessage = (event) => {
      const evt: ModelStatusEvent = JSON.parse(event.data);
      handleStatusChange(evt, setModels);
    };
    return () => ws.close();
  }, []);

  return models;
};
```

The `handleStatusChange` function (not shown inline for brevity) implements an optimistic update strategy: it applies the event payload to the local model state immediately upon receipt, then schedules a background reconciliation request to fetch the authoritative state from the scoring engine. This approach ensures that the UI feels instantaneous while maintaining eventual consistency with the backend. The WebSocket connection automatically reconnects with exponential backoff (starting at 1 second, capping at 30 seconds) if the connection drops, ensuring resilience against transient network failures.

7.3 Batch Translation with Multi-Model Orchestration

Enterprise translation workflows frequently involve processing large documents that span multiple content domains. A single pharmaceutical regulatory submission, for example, may contain chemical nomenclature (technical domain), patient safety warnings (medical domain), and legal disclaimers (legal domain). Processing such a document through a single general-purpose model produces suboptimal results because no single model excels across all domain categories. The batch translation system described in this section addresses this challenge through domain-aware chunking and intelligent model routing, assigning each content segment to the model best suited for its specific domain.

7.3.1 Domain-Aware Document Chunking

The batch translation pipeline begins by decomposing the input document into `TranslationChunk` objects. Each chunk is classified into one of five domain categories: `legal`, `medical`, `technical`, `creative`, or `general`. Classification employs a multi-modal detection strategy that combines heuristic pattern matching with a lightweight machine learning classifier. Legal content is identified through regex patterns that match legalese signatures such as "hereinafter," "pursuant to," "witnesseth," and standard legal clause numbering formats (§1.2(a)). Medical content is detected by cross-referencing terminology against a curated medical terminology database containing 47,000 terms spanning ICD-10 codes, drug names, anatomical references, and clinical procedure identifiers. Technical content is identified by the presence of code blocks, structured data formats (JSON, XML, YAML), and high concentrations of domain-specific acronyms. Creative content is recognized through narrative pattern analysis—detection of dialogue formatting, descriptive prose density, and literary device markers. Content that fails all specialized domain tests, or falls below a confidence threshold, is classified as `general`.

7.3.2 Intelligent Chunk Routing

Once chunks are classified, the routing engine assigns each chunk to the most appropriate available model. The routing decision considers three factors in priority order: (1) domain capability match, (2) current model score, and (3) cost efficiency. A chunk classified as `legal` is routed to the highest-scoring model that possesses the `CapDomainLegal` capability. If no legal-specialized model is available (for example, due to provider outage), the system falls back to the highest-scoring general-capable model. This fallback behavior ensures continuity of service while transparently degrading quality expectations. The routing strategy is abstracted behind the `RoutingStrategy` interface, enabling experimentation with different allocation algorithms (greedy, round-robin, load-balanced, cost-optimized) without modifying the core orchestration logic.

The Go data structures in Code Block 3 define the complete domain model for batch translation jobs.

Code Block 3: BatchTranslationJob

```

type BatchTranslationJob struct {
    ID          string
    SourceLang  string
    TargetLang  string
    Chunks      []TranslationChunk
    Assignments []ModelAssignment
    Strategy    RoutingStrategy
    Status      JobStatus
}

type TranslationChunk struct {
    ID          string
    Content     string
    WordCount   int
    Domain      string // detected via classifier
}

type ModelAssignment struct {
    ChunkID  string
    ModelID  string
    Status   AssignmentStatus
    Result   *TranslationResult
}

type RoutingStrategy interface {
    AssignChunks(chunks []TranslationChunk, models
        []VerifiedLLMClient) []ModelAssignment
}

```

The BatchTranslationJob struct serves as the aggregate root, tracking the job's lifecycle from Pending through Chunking, Routing, Translating, Aggregating, and finally Completed or Failed. The TranslationChunk struct carries the domain classification result alongside the raw content and word count, which feeds into cost estimation and progress reporting. Each ModelAssignment records which model was assigned to a specific chunk, the current status of that assignment (Pending, InProgress, Completed, Failed), and a pointer to the translation result once available. The RoutingStrategy interface enables pluggable routing algorithms, with a default CapabilityAwareRouter that implements the domain-match-with-fallback logic described above.

7.3.3 Chunk Routing Strategy Reference

The routing matrix in Table 1 documents the complete decision logic for chunk-to-model assignment. Each content type maps to a detection method, a preferred capability requirement, and a fallback capability for degraded-mode operation.

Table 1: Chunk routing strategy

Content Type	Detection Method	Preferred Capability	Fallback
Legal	Regex (legalese patterns)	CapDomainLegal	CapGeneral

Content Type	Detection Method	Preferred Capability	Fallback
Medical	Medical terminology DB	CapDomainMedical	CapGeneral
Technical	Code blocks, acronyms	CapDomainTechnical	CapGeneral
Creative	Narrative patterns	CapDomainCreative	CapGeneral
General	Default / short text	Balanced score	Any available

The `Balanced score` entry for general content indicates that the routing engine selects the model with the highest composite score across all capability components, rather than requiring a specific domain tag. This ensures that general content benefits from the best available model even when no specialized model exists. The fallback chain always terminates at `CapGeneral` or `Any available`, guaranteeing that every chunk receives a translation assignment as long as at least one model is operational.

7.4 Translation Quality Feedback Loop

The most reliable measure of a translation model's quality is the judgment of the humans who consume its output. The translation quality feedback loop closes the circuit between automated scoring and subjective human evaluation, creating a composite quality signal that reflects both machine-measured accuracy and human-perceived utility. This feedback loop operates continuously: every translation output carries an invitation for user rating, every rating is incorporated into the model's capability score through a weighted moving average, and the updated scores flow back into the model selection and routing systems within minutes.

7.4.1 User Rating Interface

After each translation is delivered, the user interface presents a non-intrusive rating widget inviting the user to assign 1–5 stars and optionally provide free-text feedback. The rating prompt appears in a collapsible panel adjacent to the translation output, minimizing disruption to the user's workflow. Five-star ratings indicate excellent quality requiring no further attention. Four-star ratings signal minor issues—perhaps a single awkward phrasing or terminology choice. Three-star and below trigger an expanded feedback form asking the user to categorize the problem (accuracy, fluency, terminology, formatting, cultural appropriateness). These categorizations feed into granular capability component scores, enabling the system to distinguish between a model that struggles with medical terminology but excels at grammatical fluency versus one with the opposite profile.

7.4.2 Automatic Quality Metrics

In parallel with user ratings, the system computes automatic quality metrics for every translation. The BLEU (Bilingual Evaluation Understudy) score measures n-gram overlap between the model output and reference translations from the HelixTranslate validation corpus. The COMET (Cross-lingual Optimized Metric for Evaluation of Translation) score provides a neural evaluation that correlates more

strongly with human judgments than BLEU by leveraging cross-lingual sentence embeddings. Semantic similarity is computed using cosine distance between vector embeddings of the source text and back-translated output, detecting meaning drift that lexical metrics might miss. Perplexity measures the model's confidence in its output, with unusually high perplexity signaling potential hallucination or exposure to out-of-distribution content. These automatic metrics are computed asynchronously in a background worker queue, with results typically available within 2–5 minutes of translation completion.

7.4.3 Composite Score Computation

The composite quality engine synthesizes three independent signal sources into a unified quality score using a weighted formula. User ratings contribute 40% of the composite weight, reflecting the primacy of human judgment. Automatic metrics (BLEU, COMET, semantic similarity) contribute 35%, providing an objective baseline that operates independently of user engagement. Challenge pass rate—derived from the Phase 4 verification challenge system—contributes 25%, capturing the model's performance on structured, adversarial test cases. The `SubmitRating` method in Code Block 4 demonstrates how a newly submitted user rating triggers an immediate capability score update.

Code Block 4: UserRating

```
type UserRating struct {
    TranslationID string `json:"translation_id"`
    ModelID       string `json:"model_id"`
    Rating        int    `json:"rating" // 1-5
    Feedback      string `json:"feedback"`
    CreatedAt     time.Time `json:"created_at"`
}

func (e *ScoringEngine) SubmitRating(ctx context.Context, rating
    UserRating) error {
    // Weighted moving average: 40% user ratings + 35% auto
    // metrics + 25% challenge pass rate
    current, _ := e.GetModelScore(ctx, rating.ModelID)
    newCapability := 0.4*float64(rating.Rating)*2.0 +
        0.35*current.Components.Capability +
        0.25*current.ChallengePassRate
    return e.UpdateCapabilityScore(ctx, rating.ModelID,
        newCapability)
}
```

The rating scaling logic (`float64(rating.Rating) * 2.0`) maps the 1–5 star scale onto the 0–10 score scale used by the composite engine, ensuring dimensional consistency across signal sources. The `UpdateCapabilityScore` method persists the new score to the model registry and emits a `score_changed` WebSocket event, which propagates the update to all connected dashboards within milliseconds. The use of a weighted moving average (rather than simple averaging) ensures that new ratings have immediate impact while historical data still influences the score, preventing erratic swings from single outlier ratings.

7.4.4 Signal Source Reference

Table 2 documents the complete signal inventory, including each source's relative weight, the specific metrics it contributes, and its characteristic latency profile.

Table 2: Quality signals

Signal Source	Weight	Metric	Latency
User ratings	40%	1-5 stars	Real-time
Auto metrics	35%	BLEU, COMET, semantic sim	Minutes
Challenge pass	25%	Pass rate 0-100%	Hours

The latency column is operationally significant because it determines how quickly each signal reflects changes in model behavior. User ratings provide the fastest feedback but depend on user engagement—models serving low-traffic language pairs may accumulate ratings slowly. Automatic metrics provide consistent, engagement-independent coverage with a moderate delay. Challenge pass rates update the slowest (reflecting the batch nature of verification challenge execution) but capture adversarial robustness that neither user ratings nor automatic metrics can reliably measure.

7.4.5 Composite Quality Engine Implementation

The full composite quality engine is implemented in Go as shown in Code Block 5. The engine encapsulates the three weight constants, enforces their sum to 1.0 during initialization, and exposes a single `CalculateQuality` method that retrieves the component scores and computes the weighted composite.

Code Block 5: Composite quality engine

```
type CompositeQualityEngine struct {
    userRatingWeight float64 // 0.40
    autoMetricWeight float64 // 0.35
    challengePassWeight float64 // 0.25
}

func (q *CompositeQualityEngine) CalculateQuality(ctx
    context.Context, modelID string) (float64, error) {
    userScore := q.getAverageUserRating(modelID) * 2.0 // scale
        1-5 to 0-10
    autoScore := q.getAutoMetricsScore(modelID)
    challengeScore := q.getChallengePassRate(modelID) * 0.1 //
        scale 0-100 to 0-10

    composite := q.userRatingWeight*userScore +
        q.autoMetricWeight*autoScore +
        q.challengePassWeight*challengeScore
    return composite, nil
}
```

The normalization logic ensures dimensional consistency across the three inputs. User ratings on a 1–5 scale are multiplied by 2.0 to map onto the 0–10 composite scale. Challenge pass rates on a 0–100 percentage scale are multiplied by 0.1 for the same purpose. The `getAutoMetricsScore` method returns a score already normalized to the 0–10 scale by the Phase 2 scoring pipeline. The engine validates during construction that `userRatingWeight + autoMetricWeight + challengePassWeight == 1.0`, panicking on violation to prevent silent score corruption. Weight values are configurable through environment variables, enabling A/B experiments and operational adjustments without code changes.

The composite score produced by this engine feeds directly into the `VerifiedModel.score.overall` field consumed by the `ModelCard` component in Section 7.1 and the `RoutingStrategy` implementations in Section 7.3. This architectural closure ensures that user feedback, once submitted, propagates through the entire system—from the rating widget, through the composite engine, into the model registry, across the `WebSocket` stream, onto the model selection dashboard, and ultimately into the routing decisions that determine which model translates the next chunk of content. The feedback loop is fully closed, self-reinforcing, and operates without human intervention beyond the initial rating gesture.